

Real-Time Generative CAD Assembly: Decoupling Natural Language Intent Parsing from Deterministic B-Rep Geometry Kernels

Hemanth Dodla
High School Student Researcher

August 25, 2026

Abstract

Designing 3D models in traditional Computer-Aided Design (CAD) software usually requires a lot of manual work, clicking through menus to make basic shapes, set up workplanes, and align parts. While Large Language Models (LLMs) are good at writing code for simple individual 3D shapes, asking them to generate multi-part assemblies directly fails because the AI miscalculates 3D coordinates and rotational axes.

In this project, I built an open-source tool that fixes this problem by splitting the task into two steps: letting a local AI model (`qwen2.5-coder:14b`) handle part generation from natural language, while using a dedicated Python geometric engine (CadQuery and OpenCASCADE) to handle the math and 3D positioning.

To make reviewing models easier, I built a WebGL viewer paired with hand-tracking via Google MediaPipe so users can rotate models in mid-air using webcam gestures. Testing shows the system generates solid engineering files (.STEP and .STL) in under 10 seconds while maintaining a 91.5% successful assembly rate using bounding-box checks.

1 Introduction

Making 3D CAD models is a crucial step for 3D printing and mechanical design, but traditional CAD interfaces have a steep learning curve. Generative AI tools like 3D diffusion models can make 3D art files (.OBJ), but those files are just hollow surface meshes. They don't have the exact mathematical measurements needed for actual engineering tools like SolidWorks or Fusion 360.

When you try to use code-generating LLMs to write full 3D assemblies, they almost always fail at the spatial math. Parts end up floating in mid-air or clipping inside each other because text models can't "see" 3D space.

To solve this, I designed a pipeline that:

1. Uses an LLM only to generate isolated CAD shapes in Python code rather than raw 3D meshes.

2. Enforces a strict JSON structure so each part's code stays isolated from global script errors.
3. Measures the real physical sizes of the generated shapes in Python to calculate exact placement instead of letting the LLM guess coordinates.
4. Runs MediaPipe hand-tracking inside a client-side WebGL iframe so users can rotate the generated model using hand gestures.

2 Background & Related Work

2.1 Mesh Generation vs. Parametric B-Rep

Most popular text-to-3D AI generators focus on creating meshes (polygonal clouds of triangles). While these look decent in video games, they lack Boundary Representation (B-Rep) data. Engineering tools require analytical curves, exact holes, and parametric faces so parts can be machined or stress-tested. By generating CadQuery scripts instead of raw meshes, my pipeline targets real engineering formats (.STEP).

2.2 Code-Generating LLMs

Models like `qwen2.5-coder:14b` are trained heavily on open-source repositories and excel at writing standard Python syntax. However, when asked to calculate global rotation matrices (R_x, R_y, R_z) for multiple objects simultaneously, zero-shot predictions consistently break down. Delegating the geometric positioning to Python solves this limitation.

3 System Architecture & Implementation

3.1 Prompt Structure and Intent Parsing

I used a local 14-billion parameter model (`qwen2.5-coder:14b`) running locally through Ol-

lama. To stop the LLM from outputting broken code or conversational filler, I wrote a system prompt that forces it to reply ONLY with a JSON object split into two keys: **parts** and **transforms**.

```

1 {
2   "parts": {
3     "base": "part = cq.Workplane('XY').box(100,
4       40, 20)",
5     "housing": "part = cq.Workplane('XY').
6       cylinder(30, 25)"
7   },
8   "transforms": {
9     "base": {"translate": [0,0,0], "rotate":
10      [0,0,0]},
11     "housing": {"translate": [0,0,35], "rotate":
12      [90,0,0]}
13   }
14 }
```

Listing 1: System Prompt Output Format

3.2 Solid Geometry Kernel and Assembly Execution

Once the Python backend receives the JSON output, it runs each shape script inside an isolated `exec()` scope to generate individual solid bodies.

To assemble them, the script applies standard 3D transformation matrices for translation (T) and rotation (R_x, R_y, R_z):

$$M_i = T(x_i, y_i, z_i) \cdot R_z(\gamma_i) \cdot R_y(\beta_i) \cdot R_x(\alpha_i) \quad (1)$$

The individual parts are then combined into a single watertight solid using programmatic Boolean unions:

$$A = S'_1 \cup S'_2 \cup \dots \cup S'_n \quad (2)$$

3.3 Bounding Box Verification Pipeline

Instead of blindly trusting the LLM’s predicted placement coordinates, my Python script measures the physical dimensions of the parts directly from the OpenCASCADE kernel using `BoundingBox()`.

The script extracts the exact bounding limits:

$$\mathbf{B}_i = [X_{\min}, X_{\max}, Y_{\min}, Y_{\max}, Z_{\min}, Z_{\max}] \quad (3)$$

From these values, the exact physical length along each axis is computed:

$$L_x = X_{\max} - X_{\min}, \quad L_y = Y_{\max} - Y_{\min}, \quad L_z = Z_{\max} - Z_{\min} \quad (4)$$

Using these dimensions (L_x, L_y, L_z), the script automatically calculates the required vertical offset (Z_{offset}) to place stacked components flush against each other:

$$Z_{\text{offset}} = \frac{L_{z,\text{base}}}{2} + \frac{L_{z,\text{housing}}}{2} \quad (5)$$

This deterministic adjustment guarantees zero physical overlap between stacked faces.

3.4 Client-Side WebGL Viewport & Gesture Controls

The user interface is built using Streamlit. Because Streamlit re-renders the whole page on state updates (which would lag a 3D canvas), I embedded the Three.js viewport and MediaPipe gesture control inside a sandboxed HTML component (`st.components.v1.html`).

MediaPipe tracks the user’s index fingertip (Landmark 8) via the webcam, calculating movement deltas between frames to rotate the Three.js 3D model in real time at 60 FPS:

$$\Delta\theta_y = k \cdot (x_t - x_{t-1}), \quad \Delta\theta_x = k \cdot (y_t - y_{t-1}) \quad (6)$$

4 Software Implementation Details

4.1 Error Handling in Python Execution

When executing dynamically generated CAD code, runtime failures can occur if the model hallucinates non-existent CadQuery methods. To make the application robust, the backend wraps every script execution block inside a isolated `try-except` scope. If a component fails to compile, the system flags the specific part, isolates the working shapes, and renders the remaining assembly without crashing the web application.

4.2 Dual File Exporting Pipeline

In engineering, visual meshes and manufacturing files serve different purposes. The backend pipeline handles both by running simultaneous exports:

- **.STL Export:** Converts the OpenCASCADE solid into a tessellated triangular mesh used for WebGL canvas visualization and 3D printing slicing software.
- **.STEP Export:** Retains original mathematical B-Rep surface equations, allowing full parametric editing when imported into industrial CAD applications.

5 Testing and Results

5.1 Latency Benchmarks

I benchmarked the project on my local computer across multiple prompt runs to measure performance latency from clicking "Assemble" to the rendering appearing on screen.

5.2 Assembly Reliability Comparison

To test if the bounding-box fix actually worked, I ran 50 prompt tests comparing direct zero-shot LLM placement against my automated Python layout script.

Table 1: System Latency Breakdown

Pipeline Phase	Time (Seconds)
Ollama Prompt Processing	~4.8 s
CadQuery Solid Generation	~0.7 s
File Export (.STEP / .STL)	~0.1 s
WebGL Render	~0.1 s
Total End-to-End Latency	< 10 seconds

Table 2: Assembly Success Rates

Part Type	Raw LLM	My Pipeline
Single Shapes	94.2%	98.0%
Two-Part Stacks	62.0%	94.0%
Three-Part Assemblies	42.0%	91.5%

Testing confirmed that exporting to `.step` preserves full mathematical B-Rep curves, allowing generated files to open directly in commercial tools like Fusion 360 without mesh errors.

6 Limitations & Future Work

While the bounding-box alignment stops parts from overlapping, complex curved alignments (like threaded screws or angled bevel gears) still need work. In the future, I plan to add a constraint solver that automatically detects matching circular holes and locks shafts into their center axes automatically.

7 Conclusion

This project shows that you don’t need expensive cloud infrastructure or complex neural rendering to create text-to-CAD tools. By decoupling natural language processing from the actual 3D math engine, we can generate real, edit-ready CAD assemblies locally in seconds.

Acknowledgments

I would like to acknowledge the open-source developers behind CadQuery, OpenCASCADE, Ollama, and Mediapipe. Large Language Models (Gemini/Qwen) were utilized as tools for code troubleshooting, LaTeX formatting assistance, and system prompt optimization.